

Referência do Diretório aidlc-docs/ Gerado

Author: Raja SP, AWS Labs

Source: <https://github.com/aws-labs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [generated-docs-reference.pt-br.pdf](#)

Quando você executa o workflow do AI-DLC, todos os artefatos de documentação são gerados dentro de um diretório `aidlc-docs/` na raiz do seu workspace. Os arquivos exatos criados dependem do tipo de projeto (greenfield vs brownfield), da complexidade e de quais etapas o workflow executa ou ignora.

Abaixo está a estrutura totalmente preenchida mostrando todos os possíveis arquivos em todas as fases e etapas. Os arquivos condicionais são anotados com notas indicando quando aparecem.

```

aidlc-docs/
├─ aidlc-state.md # Rastreador de estado do workflow — informações do projeto, progresso das etapas, status atual
├─ audit.md # Trilha de auditoria completa — cada input do usuário, resposta da IA e aprovação com timestamps
|
├─ inception/ # ● FASE INCEPTION — determina O QUE construir e POR QUÊ
|   └─ plans/
|       └─ execution-plan.md # Visualização do workflow e decisões de execução da fase (sempre criado)
|           └─ story-generation-plan.md # Metodologia de desenvolvimento de histórias e perguntas (se User Stories for executado)
|               └─ user-stories-assessment.md # Avaliação de se as user stories agregam valor (se User Stories for executado)
|                   └─ application-design-plan.md # Plano de design de componentes e serviços com perguntas (se Application Design for executado)
|                       └─ unit-of-work-plan.md # Plano de decomposição do sistema com perguntas (se Units Generation for executado)
|
|   └─ reverse-engineering/ # Criado apenas para projetos brownfield (codebase existente detectada)
|       └─ business-overview.md # Contexto de negócio, transações e dicionário
|           └─ architecture.md # Diagramas de arquitetura do sistema, descrições de componentes, fluxo de dados
|               └─ code-structure.md # Sistema de build, classes/módulos principais, padrões de design, inventário de arquivos
|                   └─ api-documentation.md # APIs REST, APIs internas e modelos de dados
|                       └─ component-inventory.md # Inventário de todos os pacotes por tipo (aplicação, infraestrutura, compartilhado, teste)
|                           └─ technology-stack.md # Linguagens, frameworks, infraestrutura, ferramentas de build, ferramentas de teste
|                               └─ dependencies.md # Grafos e relacionamentos de dependência internos e externos
|                                   └─ code-quality-assessment.md # Cobertura de testes, indicadores de qualidade de código, dívida técnica, padrões
|                                       └─ reverse-engineering-timestamp.md # Metadados da análise e checklist de artefatos
|
|   └─ requirements/ # Requisitos funcionais e não-funcionais com análise de intenção (sempre criado)
|       └─ requirements.md
|           └─ requirement-verification-questions.md # Perguntas de esclarecimento com tags [Answer]: para input do usuário (sempre criado)
|
|   └─ user-stories/ # Criado apenas se a etapa User Stories for executada
|       └─ stories.md # User stories seguindo os critérios INVEST com critérios de aceitação
|           └─ personas.md # Arquétipos de usuário, características e mapeamentos persona-para-história

```

```

| |
| | └─ application-design/ # Criado apenas se Application Design e/ou Units Generation forem executados
| |   └─ application-design.md # Documento de design consolidado (se Application Design for executado)
| |   └─ components.md # Definições, responsabilidades e interfaces de componentes
| |   └─ component-methods.md # Assinaturas de métodos, propósitos e tipos de entrada/saída
| |   └─ services.md # Definições, responsabilidades e padrões de orquestração de serviços
| |   └─ component-dependency.md # Matriz de dependências e padrões de comunicação entre componentes
| |   └─ unit-of-work.md # Definições e responsabilidades das unidades (se Units Generation for executado)
| |   └─ unit-of-work-dependency.md # Matriz de dependências entre unidades (se Units Generation for executado)
| |   └─ unit-of-work-story-map.md # Mapeamento de user stories para as unidades (se Units Generation for executado)
| |
| └─ construction/ # ● FASE CONSTRUCTION – determina COMO construir
| | └─ plans/
| | | └─ {unit-name}-functional-design-plan.md # Plano de design de lógica de negócio com perguntas (por unidade, se Functional Design for executado)
| | | └─ {unit-name}-nfr-requirements-plan.md # Plano de avaliação de NFR com perguntas (por unidade, se NFR Requirements for executado)
| | | └─ {unit-name}-nfr-design-plan.md # Plano de padrões de design NFR com perguntas (por unidade, se NFR Design for executado)
| | | └─ {unit-name}-infrastructure-design-plan.md # Plano de mapeamento de infraestrutura com perguntas (por unidade, se Infrastructure Design for executado)
| | | └─ {unit-name}-code-generation-plan.md # Etapas detalhadas de geração de código com checkboxes (por unidade, sempre criado)
| | |
| | └─ {unit-name}/ # Artefatos por unidade – um diretório por unidade de trabalho
| | | └─ functional-design/ # Criado apenas se Functional Design for executado para esta unidade
| | | | └─ business-logic-model.md # Lógica de negócio detalhada e algoritmos
| | | | └─ business-rules.md # Regras de negócio, lógica de validação e restrições
| | | | └─ domain-entities.md # Modelos de domínio com entidades e relacionamentos
| | | | └─ frontend-components.md # Hierarquia de componentes de UI, props, estado, interações (se a unidade tiver frontend)
| | |
| | └─ nfr-requirements/ # Criado apenas se NFR Requirements for executado para esta unidade
| | | └─ nfr-requirements.md # Requisitos de escalabilidade, performance, disponibilidade e segurança
| | | └─ tech-stack-decisions.md # Escolhas tecnológicas e justificativas
| | |
| | |

```

```

| | └─ nfr-design/                                # Criado apenas se NFR Design fo
r executado para esta unidade
| | | └─ nfr-design-patterns.md                    # Padrões de resiliência, escala
bilidade, performance e segurança
| | | └─ logical-components.md                     # Componentes lógicos de infraes
trutura (filas, caches, etc.)
| | |
| | └─ infrastructure-design/                      # Criado apenas se Infrastructur
e Design for executado para esta unidade
| | | └─ infrastructure-design.md                  # Mapeamentos de serviços cloud
e componentes de infraestrutura
| | | └─ deployment-architecture.md                # Modelo de deploy, rede, config
uração de escalabilidade
| | |
| | └─ code/                                       # Resumos em Markdown do código
gerado (sempre criado por unidade)
| | └─ *.md                                       # Resumos de geração de código
(o código real vai para a raiz do workspace)
| |
| └─ shared-infrastructure.md                      # Infraestrutura compartilhada e
ntre unidades (se aplicável)
| |
| └─ build-and-test/                              # Sempre criado após todas as un
idades completarem a geração de código
|   └─ build-instructions.md                      # Pré-requisitos, etapas de buil
d, solução de problemas
|   └─ unit-test-instructions.md                  # Comandos de execução de testes
unitários e resultados esperados
|   └─ integration-test-instructions.md           # Cenários de teste de integraçã
o, configuração e execução
|   └─ performance-test-instructions.md           # Configuração e execução de tes
tes de carga/stress (se existirem NFRs de performance)
|   └─ contract-test-instructions.md              # Validação de contrato de API e
ntre serviços (se microserviços)
|   └─ security-test-instructions.md              # Varredura de vulnerabilidades
e testes de segurança (se existirem NFRs de segurança)
|   └─ e2e-test-instructions.md                  # Testes de fluxo de usuário end
-to-end (se aplicável)
|   └─ build-and-test-summary.md                  # Status geral de build, resulta
dos de testes e avaliação de prontidão
|
└─ operations/                                    # 🟡 FASE OPERATIONS – placeholder
para expansão futura

```

Notas

- `{unit-name}` é substituído pelo nome real da unidade (ex.: `api-service`, `frontend-app`, `data-processor`). Para projetos de unidade única, tipicamente há um diretório de unidade em `construction/`.

- Para projetos de unidade única mais simples, o modelo pode simplificar a nomenclatura — por exemplo, `construction/plans/code-generation-plan.md` em vez de `construction/plans/{unit-name}-code-generation-plan.md`, ou colocar `application-design.md` como um único arquivo consolidado sem os arquivos individuais de componentes.
- O diretório `build-and-test/` sempre inclui `build-and-test-summary.md`. Os arquivos individuais de instrução (`build-instructions.md`, `unit-test-instructions.md`, `integration-test-instructions.md`, etc.) são gerados com base na complexidade do projeto e nas necessidades de teste.
- Os planos em `inception/plans/` e `construction/plans/` contêm tags `[Answer]:` onde os usuários fornecem input, e checkboxes `[ ]`/`[x]` que rastreiam o progresso de execução.
- O código da aplicação nunca é colocado dentro de `aidlc-docs/` — ele vai para a raiz do workspace. Apenas documentação em Markdown fica aqui.
- O arquivo `audit.md` é somente de adição e captura cada interação com timestamps ISO 8601.
- O arquivo `aidlc-state.md` rastreia quais etapas foram concluídas, ignoradas ou estão em andamento, junto com a configuração de extensão.